
title: FreeBSD Architecture Handbook authors: - author: The FreeBSD Documentation Project copyright: 2000-2006, 2012-2023 The FreeBSD Documentation Project description: For FreeBSD system developers. This book covers the architectural details of many important FreeBSD kernel subsystems trademarks: ["freebsd", "apple", "microsoft", "unix", "general"] tags: ["Arch Handbook", "FreeBSD"] add_split_page_link: true ---

FreeBSD Architecture Handbook

Abstract

Welcome to the FreeBSD Architecture Handbook. This manual is a *work in progress* and is the work of many individuals. Many sections do not yet exist and some of those that do exist need to be updated. If you are interested in helping with this project, send email to the [FreeBSD documentation project mailing list](#).

The latest version of this document is always available from the [FreeBSD World Wide Web server](#). It may also be downloaded in a variety of formats and compression options from the [FreeBSD download server](#) or one of the numerous [extref:https://docs.freebsd.org/en/books/handbook/mirrors](https://docs.freebsd.org/en/books/handbook/mirrors)[mirror sites, mirrors].

title: Part I. Kernel prev: books/arch-handbook next: books/arch-handbook/boot showBookMenu: true weight: 1 params: path: "/books/arch-handbook/parti/" BundleType: branch ---

Kernel

title: Chapter 1. Bootstrapping and Kernel Initialization prev: books/arch-handbook/parti next: books/arch-handbook/locking description: Bootstrapping and Kernel Initialization tags: ["boot", "BIOS", "kernel", "MBR", "FreeBSD"] showBookMenu: true weight: 2 params: path: "/books/arch-handbook/boot/" ---

1. Bootstrapping and Kernel Initialization

1.1. Synopsis

This chapter is an overview of the boot and system initialization processes, starting from the BIOS (firmware) POST, to the first user process creation. Since the initial steps of system startup are very architecture dependent, the IA-32 architecture is used as an example. But the AMD64 and ARM64 architectures are much more important and compelling examples and should be explained in the near future according to the topic of this document.

The FreeBSD boot process can be surprisingly complex. After control is passed from the BIOS, a considerable amount of low-level configuration must be done before the kernel can be loaded and executed. This setup must be done in a simple and flexible manner, allowing the user a great deal of customization possibilities.

1.2. Overview

The boot process is an extremely machine-dependent activity. Not only must code be written for every computer architecture, but there may also be multiple types of booting on the same architecture. For example, a directory listing of `stand` reveals a great amount of architecture-dependent code. There is a directory for each of the various supported architectures. FreeBSD supports the CSM boot standard (Compatibility Support Module). So CSM is supported (with both GPT and MBR partitioning support) and UEFI booting (GPT is totally supported, MBR is mostly supported). It also supports loading files from `ext2fs`, `MSDOS`, `UFS` and `ZFS`. FreeBSD also supports the boot environment feature of `ZFS` which allows the HOST OS to communicate details about what to boot that go beyond a simple partition as was possible in the past. But UEFI is more relevant than the CSM these days. The example that follows shows booting an x86 computer from an MBR-partitioned hard drive with the FreeBSD `boot0` multi-boot loader stored in the very first sector. That boot code starts the FreeBSD three-stage boot process.

The key to understanding this process is that it is a series of stages of increasing complexity. These stages are `boot1`, `boot2`, and `loader` (see `man:boot[8]` for more detail). The boot system executes each stage in sequence. The last stage, `loader`, is responsible for loading the FreeBSD kernel. Each stage is examined in the following sections.

Here is an example of the output generated by the different boot stages. Actual output may differ from machine to machine:

FreeBSD Component	Output (may vary)
boot0	<pre> F1 FreeBSD F2 BSD F5 Disk 2 </pre>
boot2 ^[1]	<pre> >>FreeBSD/x86 B00T Default: 0:ad(0p4)/boot/loader boot: </pre>
loader	<pre> BTX loader 1.00 BTX version is 1.02 Consoles: internal video/keyboard BIOS drive C: is disk0 BIOS 639kB/2096064kB available memory FreeBSD/x86 bootstrap loader, Revision 1.1 Console internal video/keyboard (root@releng1.nyi.freebsd.org, Fri Apr 9 04:04:45 UTC 2021) Loading /boot/defaults/loader.conf /boot/kernel/kernel text=0xed9008 data =0x117d28+0x176650 syms=[0x8+0x137988+0x8+0x1515f8] </pre>

kernel

```
Copyright (c) 1992-2021 The FreeBSD Project.
Copyright (c) 1979, 1980, 1983, 1986, 1988, 1989,
1991, 1992, 1993, 1994
    The Regents of the University of
California. All rights reserved.
FreeBSD is a registered trademark of The FreeBSD
Foundation.
FreeBSD 13.0-RELEASE 0 releng/13.0-n244733-
ea31abc261f: Fri Apr  9 04:04:45 UTC 2021

root@releng1.nyi.freebsd.org:/usr/obj/usr/src/i386.
i386/sys/GENERIC i386
FreeBSD clang version 11.0.1
(git@github.com:llvm/llvm-project.git llvmorg-
11.0.1-0-g43ff75f2c3fe)
```

1.3. The BIOS

When the computer powers on, the processor's registers are set to some predefined values. One of the registers is the *instruction pointer* register, and its value after a power on is well defined: it is a 32-bit value of `0xffffffff0`. The instruction pointer register (also known as the Program Counter) points to code to be executed by the processor. Another important register is the `cr0` 32-bit control register, and its value just after a reboot is `0`. One of `cr0`'s bits, the PE (Protection Enabled) bit, indicates whether the processor is running in 32-bit protected mode or 16-bit real mode. Since this bit is cleared at boot time, the processor boots in 16-bit real mode. Real mode means, among other things, that linear and physical addresses are identical. The reason for the processor not to start immediately in 32-bit protected mode is backwards compatibility. In particular, the boot process relies on the services provided by the BIOS, and the BIOS itself works in legacy, 16-bit code.

The value of `0xffffffff0` is slightly less than 4 GB, so unless the machine has 4 GB of physical memory, it cannot point to a valid memory address. The computer's hardware translates this address so that it points to a BIOS memory block.

The BIOS (Basic Input Output System) is a chip on the motherboard that has a relatively small amount of read-only memory (ROM). This memory contains various low-level routines that are specific to the hardware supplied with the motherboard. The processor will first jump to the address `0xffffffff0`, which really resides in the BIOS's memory. Usually this address contains a jump instruction to the BIOS's POST routines.

The POST (Power On Self Test) is a set of routines including the memory check, system bus check, and other low-level initialization so the CPU can set up the computer properly. The important step of this stage is determining the boot device. Modern BIOS implementations permit the selection of a boot device, allowing booting from a floppy, CD-ROM, hard disk, or other devices.

The very last thing in the POST is the `INT 0x19` instruction. The `INT 0x19` handler reads 512 bytes from the first sector of boot device into the memory at address `0x7c00`. The term *first sector* originates from hard drive architecture, where the magnetic plate is divided into a number of cylindrical tracks. Tracks are numbered, and every track is divided into a number (usually 64) of sectors. Track numbers start at 0, but sector numbers start from 1. Track 0 is the outermost on the magnetic plate, and sector 1, the first sector, has a special purpose. It is also called the MBR, or Master Boot Record. The remaining sectors on the first track are never used.

This sector is our boot-sequence starting point. As we will see, this sector contains a copy of our `boot0` program. A jump is made by the BIOS to address `0x7c00` so it starts executing.

1.4. The Master Boot Record (`boot0`)

After control is received from the BIOS at memory address `0x7c00`, `boot0` starts executing. It is the first piece of code under FreeBSD control. The task of `boot0` is quite simple: scan the partition table and let the user choose which partition to boot from. The Partition Table is a special, standard data structure embedded in the MBR (hence embedded in `boot0`) describing the four standard PC "partitions". `boot0` resides in the filesystem as `/boot/boot0`. It is a small 512-byte file, and it is exactly what FreeBSD's installation procedure wrote to the hard disk's MBR if you chose the "bootmanager" option at installation time. Indeed, `boot0` is the MBR.

As mentioned previously, we're calling the BIOS `INT 0x19` to load the MBR (`boot0`) into memory at address `0x7c00`. The source file for `boot0` can be found in `stand/i386/boot0/boot0.S` - which is an awesome piece of code written by Robert Nordier.

A special structure starting from offset `0x1be` in the MBR is called the *partition table*. It has four records of 16 bytes each, called *partition records*, which represent how the hard disk is partitioned, or, in FreeBSD's terminology, sliced. One byte of those 16 says whether a partition (slice) is bootable or not. Exactly one record must have that flag set, otherwise `boot0`'s code will refuse to proceed.

A partition record has the following fields:

- the 1-byte filesystem type
- the 1-byte bootable flag
- the 6 byte descriptor in CHS format
- the 8 byte descriptor in LBA format

A partition record descriptor contains information about where exactly the partition resides on the drive. Both descriptors, LBA and CHS, describe the same information, but in different ways: LBA (Logical Block Addressing) has the starting sector for the partition and the partition's length, while CHS (Cylinder Head Sector) has coordinates for the first and last sectors of the partition. The partition table ends with the special signature `0xaa55`.

The MBR must fit into 512 bytes, a single disk sector. This program uses low-level "tricks" like taking advantage of the side effects of certain instructions and reusing register values from previous operations to make the most out of the fewest possible instructions. Care must also be taken when handling the partition table, which is embedded in the MBR itself. For these reasons, be very careful when modifying `boot0.S`.

Note that the `boot0.S` source file is assembled "as is": instructions are translated one by one to binary, with no additional information (no ELF file format, for example). This kind of low-level control is achieved at link time through special control flags passed to the linker. For example, the text section of the program is set to be located at address `0x600`. In practice this means that `boot0` must be loaded to memory address `0x600` in order to function properly.

It is worth looking at the Makefile for `boot0` (`stand/i386/boot0/Makefile`), as it defines some of the run-time behavior of `boot0`. For instance, if a terminal connected to the serial port (COM1) is used for I/O, the macro `SIO` must be defined (`-DSIO`). `-DPXE` enables boot through PXE by pressing `F6`. Additionally, the program defines a set of *flags* that allow further modification of its behavior. All of this is illustrated in the Makefile. For example, look at the linker directives which command the linker to start the text section at address `0x600`, and to build the output file "as is" (strip out any file formatting):

```
BOOT_BOOT0_ORG?=0x600
ORG=${BOOT_BOOT0_ORG}
```

Let us now start our study of the MBR, or boot0, starting where execution begins.



Some modifications have been made to some instructions in favor of better exposition. For example, some macros are expanded, and some macro tests are omitted when the result of the test is known. This applies to all of the code examples shown.

```
start:
    cld                # String ops inc
    xorw %ax,%ax       # Zero
    movw %ax,%es       # Address
    movw %ax,%ds       # data
    movw %ax,%ss       # Set up
    movw $LOAD,%sp     # stack
```

stand/i386/boot0/boot0.S

This first block of code is the entry point of the program. It is where the BIOS transfers control. First, it makes sure that the string operations autoincrement its pointer operands (the `cld` instruction) ^[2]. Then, as it makes no assumption about the state of the segment registers, it initializes them. Finally, it sets the stack pointer register (`%sp`) to (`$LOAD` = address `0x7c00`), so we have a working stack.

The next block is responsible for the relocation and subsequent jump to the relocated code.

```
    movw %sp,%si      # Source
    movw $start,%di    # Destination
    movw $0x100,%cx    # Word count
    rep              # Relocate
    movsw            # code
    movw %di,%bp      # Address variables
    movb $0x8,%cl     # Words to clear
    rep              # Zero
    stosw            # them
    incb -0xe(%di)    # Set the S field to 1
    jmp main-LOAD+ORIGIN # Jump to relocated code
```

stand/i386/boot0/boot0.S

As boot0 is loaded by the BIOS to address `0x7c00`, it copies itself to address `0x600` and then transfers control there (recall that it was linked to execute at address `0x600`). The

source address, `0x7c00`, is copied to register `%si`. The destination address, `0x600`, to register `%di`. The number of words to copy, `256` (the program's size = 512 bytes), is copied to register `%cx`. Next, the `rep` instruction repeats the instruction that follows, that is, `movsw`, the number of times dictated by the `%cx` register. The `movsw` instruction copies the word pointed to by `%si` to the address pointed to by `%di`. This is repeated another 255 times. On each repetition, both the source and destination registers, `%si` and `%di`, are incremented by one. Thus, upon completion of the 256-word (512-byte) copy, `%di` has the value `0x600`512` = `0x800``, and `%si` has the value `0x7c00`512= 0x7e00`; we have thus completed the code *relocation*. Since the last update of this document, the copy instructions have changed in the code, so instead of the `movsb` and `stosb`, `movsw` and `stosw` have been introduced, which copy 2 bytes(1 word) in one iteration.

Next, the destination register `%di` is copied to `%bp`. `%bp` gets the value `0x800`. The value `8` is copied to `%cl` in preparation for a new string operation (like our previous `movsw`). Now, `stosw` is executed 8 times. This instruction copies a `0` value to the address pointed to by the destination register (`%di`, which is `0x800`), and increments it. This is repeated another 7 times, so `%di` ends up with value `0x810`. Effectively, this clears the address range `0x800-0x80f`. This range is used as a (fake) partition table for writing the MBR back to disk. Finally, the sector field for the CHS addressing of this fake partition is given the value 1 and a jump is made to the main function from the relocated code. Note that until this jump to the relocated code, any reference to an absolute address was avoided.

The following code block tests whether the drive number provided by the BIOS should be used, or the one stored in `boot0`.

```
main:
    testb $SETDRV,_FLAGS(%bp) # Set drive number?
#ifdef CHECK_DRIVE /* disable drive checks */
    jz save_curdrive          # no, use the default
#else
    jnz disable_update        # Yes
    testb %dl,%dl             # Drive number valid?
    js save_curdrive          # Possibly (0x80 set)
#endif
```

stand/i386/boot0/boot0.S

This code tests the `SETDRV` bit (`0x20`) in the `flags` variable. Recall that register `%bp` points to address location `0x800`, so the test is done to the `flags` variable at address `0x800-69=0x7bb`. This is an example of the type of modifications that can be done to `boot0`. The `SETDRV` flag is not set by default, but it can be set in the Makefile. When set, the drive

number stored in the MBR is used instead of the one provided by the BIOS. We assume the defaults, and that the BIOS provided a valid drive number, so we jump to `save_curdrive`.

The next block saves the drive number provided by the BIOS, and calls `putn` to print a new line on the screen.

```
save_curdrive:
    movb %dl, (%bp)      # Save drive number
    pushw %dx            # Also in the stack
#ifdef TEST /* test code, print internal bios drive */
    rolb $1, %dl
    movw $drive, %si
    call putkey
#endif
    callw putn           # Print a newline
```

stand/i386/boot0/boot0.S

Note that we assume `TEST` is not defined, so the conditional code in it is not assembled and will not appear in our executable `boot0`.

Our next block implements the actual scanning of the partition table. It prints to the screen the partition type for each of the four entries in the partition table. It compares each type with a list of well-known operating system file systems. Examples of recognized partition types are NTFS (Windows®, ID 0x7), `ext2fs` (Linux®, ID 0x83), and, of course, `ffs` / `ufs2` (FreeBSD, ID 0xa5). The implementation is fairly simple.

```
    movw $(partbl+0x4),%bx    # Partition table (+4)
    xorw %dx,%dx             # Item number

read_entry:
    movb %ch, -0x4(%bx)      # Zero active flag (ch == 0)
    btw %dx, _FLAGS(%bp)    # Entry enabled?
    jnc next_entry           # No
    movb (%bx), %al          # Load type
    test %al, %al            # skip empty partition
    jz next_entry
    movw $bootable_ids, %di   # Lookup tables
    movb $(TLEN+1), %cl      # Number of entries
    repne                     # Locate
    scasb                    # type
    addw $(TLEN-1), %di      # Adjust
    movb (%di), %cl          # Partition
```

```

    addw %cx,%di      # description
    callw putx        # Display it

next_entry:
    incw %dx          # Next item
    addb $0x10,%bl     # Next entry
    jnc read_entry    # Till done

```

stand/i386/boot0/boot0.S

It is important to note that the active flag for each entry is cleared, so after the scanning, *no* partition entry is active in our memory copy of boot0. Later, the active flag will be set for the selected partition. This ensures that only one active partition exists if the user chooses to write the changes back to disk.

The next block tests for other drives. At startup, the BIOS writes the number of drives present in the computer to address **0x475**. If there are any other drives present, boot0 prints the current drive to screen. The user may command boot0 to scan partitions on another drive later.

```

    popw %ax          # Drive number
    subb $0x80-0x1,%al # Does next
    cmpb NHRDRV,%al   # drive exist? (from BIOS?)
    jb print_drive    # Yes
    decw %ax          # Already drive 0?
    jz print_prompt   # Yes

```

stand/i386/boot0/boot0.S

We make the assumption that a single drive is present, so the jump to **print_drive** is not performed. We also assume nothing strange happened, so we jump to **print_prompt**.

This next block just prints out a prompt followed by the default option:

```

print_prompt:
    movw $prompt,%si   # Display
    callw putstr       # prompt
    movb _OPT(%bp),%dl  # Display
    decw %si           # default
    callw putkey       # key
    jmp start_input    # Skip beep

```

stand/i386/boot0/boot0.S

Finally, a jump is performed to `start_input`, where the BIOS services are used to start a timer and for reading user input from the keyboard; if the timer expires, the default option will be selected:

```
start_input:
    xorb %ah,%ah      # BIOS: Get
    int $0x1a         # system time
    movw %dx,%di      # Ticks when
    addw _TICKS(%bp),%di # timeout
read_key:
    movb $0x1,%ah     # BIOS: Check
    int $0x16         # for keypress
    jnz got_key       # Have input
    xorb %ah,%ah      # BIOS: int 0x1a, 00
    int $0x1a         # get system time
    cmpw %di,%dx      # Timeout?
    jb read_key       # No
```

stand/i386/boot0/boot0.S

An interrupt is requested with number `0x1a` and argument `0` in register `%ah`. The BIOS has a predefined set of services, requested by applications as software-generated interrupts through the `int` instruction and receiving arguments in registers (in this case, `%ah`). Here, particularly, we are requesting the number of clock ticks since last midnight; this value is computed by the BIOS through the RTC (Real Time Clock). This clock can be programmed to work at frequencies ranging from 2 Hz to 8192 Hz. The BIOS sets it to 18.2 Hz at startup. When the request is satisfied, a 32-bit result is returned by the BIOS in registers `%cx` and `%dx` (lower bytes in `%dx`). This result (the `%dx` part) is copied to register `%di`, and the value of the `TICKS` variable is added to `%di`. This variable resides in `boot0` at offset `_TICKS` (a negative value) from register `%bp` (which, recall, points to `0x800`). The default value of this variable is `0xb6` (182 in decimal). Now, the idea is that `boot0` constantly requests the time from the BIOS, and when the value returned in register `%dx` is greater than the value stored in `%di`, the time is up and the default selection will be made. Since the RTC ticks 18.2 times per second, this condition will be met after 10 seconds (this default behavior can be changed in the Makefile). Until this time has passed, `boot0` continually asks the BIOS for any user input; this is done through `int 0x16`, argument `1` in `%ah`.

Whether a key was pressed or the time expired, subsequent code validates the selection. Based on the selection, the register `%si` is set to point to the appropriate partition entry in the partition table. This new selection overrides the previous default one. Indeed, it becomes the new default. Finally, the `ACTIVE` flag of the selected partition is set. If it

was enabled at compile time, the in-memory version of boot0 with these modified values is written back to the MBR on disk. We leave the details of this implementation to the reader.

We now end our study with the last code block from the boot0 program:

```
movw $LOAD,%bx      # Address for read
movb $0x2,%ah        # Read sector
callw intx13         # from disk
jc beep             # If error
cmpw $MAGIC,0x1fe(%bx) # Bootable?
jne beep            # No
pushw %si           # Save ptr to selected part.
callw putn          # Leave some space
popw %si            # Restore, next stage uses it
jmp *%bx            # Invoke bootstrap
```

stand/i386/boot0/boot0.S

Recall that `%si` points to the selected partition entry. This entry tells us where the partition begins on disk. We assume, of course, that the partition selected is actually a FreeBSD slice.



From now on, we will favor the use of the technically more accurate term "slice" rather than "partition".

The transfer buffer is set to `0x7c00` (register `%bx`), and a read for the first sector of the FreeBSD slice is requested by calling `intx13`. We assume that everything went okay, so a jump to `beep` is not performed. In particular, the new sector read must end with the magic sequence `0xaa55`. Finally, the value at `%si` (the pointer to the selected partition table) is preserved for use by the next stage, and a jump is performed to address `0x7c00`, where execution of our next stage (the just-read block) is started.

1.5. boot1 Stage

So far we have gone through the following sequence:

- The BIOS did some early hardware initialization, including the POST. The MBR (boot0) was loaded from absolute disk sector one to address `0x7c00`. Execution control was passed to that location.
- boot0 relocated itself to the location it was linked to execute (`0x600`), followed by a

jump to continue execution at the appropriate place. Finally, boot0 loaded the first disk sector from the FreeBSD slice to address `0x7c00`. Execution control was passed to that location.

boot1 is the next step in the boot-loading sequence. It is the first of three boot stages. Note that we have been dealing exclusively with disk sectors. Indeed, the BIOS loads the absolute first sector, while boot0 loads the first sector of the FreeBSD slice. Both loads are to address `0x7c00`. We can conceptually think of these disk sectors as containing the files boot0 and boot1, respectively, but in reality this is not entirely true for boot1. Strictly speaking, unlike boot0, boot1 is not part of the boot blocks ^[3]. Instead, a single, full-blown file, boot (/boot/boot), is what ultimately is written to disk. This file is a combination of boot1, boot2 and the **Boot Extender** (or BTX). This single file is greater in size than a single sector (greater than 512 bytes). Fortunately, boot1 occupies *exactly* the first 512 bytes of this single file, so when boot0 loads the first sector of the FreeBSD slice (512 bytes), it is actually loading boot1 and transferring control to it.

The main task of boot1 is to load the next boot stage. This next stage is somewhat more complex. It is composed of a server called the "Boot Extender", or BTX, and a client, called boot2. As we will see, the last boot stage, loader, is also a client of the BTX server.

Let us now look in detail at what exactly is done by boot1, starting like we did for boot0, at its entry point:

```
start:
    jmp main
```

stand/i386/boot2/boot1.S

The entry point at `start` simply jumps past a special data area to the label `main`, which in turn looks like this:

```
main:
    cld                # String ops inc
    xor %cx,%cx        # Zero
    mov %cx,%es        # Address
    mov %cx,%ds        # data
    mov %cx,%ss        # Set up
    mov $start,%sp     # stack
    mov %sp,%si        # Source
    mov $MEM_REL,%di   # Destination
    incb %ch           # Word count
    rep               # Copy
```

```
movsw      # code
```

stand/i386/boot2/boot1.S

Just like boot0, this code relocates boot1, this time to memory address `0x700`. However, unlike boot0, it does not jump there. boot1 is linked to execute at address `0x7c00`, effectively where it was loaded in the first place. The reason for this relocation will be discussed shortly.

Next comes a loop that looks for the FreeBSD slice. Although boot0 loaded boot1 from the FreeBSD slice, no information was passed to it about this ^[4], so boot1 must rescan the partition table to find where the FreeBSD slice starts. Therefore it rereads the MBR:

```
mov $part4,%si      # Partition
cmpb $0x80,%dl      # Hard drive?
jnb main.4          # No
movb $0x1,%dh       # Block count
callw nread         # Read MBR
```

stand/i386/boot2/boot1.S

In the code above, register `%dl` maintains information about the boot device. This is passed on by the BIOS and preserved by the MBR. Numbers `0x80` and greater tells us that we are dealing with a hard drive, so a call is made to `nread`, where the MBR is read. Arguments to `nread` are passed through `%si` and `%dh`. The memory address at label `part4` is copied to `%si`. This memory address holds a "fake partition" to be used by `nread`. The following is the data in the fake partition:

```
part4:
.byte 0x80, 0x00, 0x01, 0x00
.byte 0xa5, 0xfe, 0xff, 0xff
.byte 0x00, 0x00, 0x00, 0x00
.byte 0x50, 0xc3, 0x00, 0x00
```

stand/i386/boot2/boot1.S

In particular, the LBA for this fake partition is hardcoded to zero. This is used as an argument to the BIOS for reading absolute sector one from the hard drive. Alternatively, CHS addressing could be used. In this case, the fake partition holds cylinder 0, head 0 and sector 1, which is equivalent to absolute sector one.

Let us now proceed to take a look at `nread`:

```

nread:
    mov $MEM_BUF,%bx      # Transfer buffer
    mov 0x8(%si),%ax       # Get
    mov 0xa(%si),%cx       # LBA
    push %cs              # Read from
    callw xread.1          # disk
    jnc return             # If success, return

```

stand/i386/boot2/boot1.S

Recall that `%si` points to the fake partition. The word ^[5] at offset `0x8` is copied to register `%ax` and word at offset `0xa` to `%cx`. They are interpreted by the BIOS as the lower 4-byte value denoting the LBA to be read (the upper four bytes are assumed to be zero). Register `%bx` holds the memory address where the MBR will be loaded. The instruction pushing `%cs` onto the stack is very interesting. In this context, it accomplishes nothing. However, as we will see shortly, `boot2`, in conjunction with the BTX server, also uses `xread.1`. This mechanism will be discussed in the next section.

The code at `xread.1` further calls the `read` function, which actually calls the BIOS asking for the disk sector:

```

xread.1:
    pushl $0x0            # absolute
    push %cx              # block
    push %ax              # number
    push %es              # Address of
    push %bx              # transfer buffer
    xor %ax,%ax           # Number of
    movb %dh,%al          # blocks to
    push %ax              # transfer
    push $0x10            # Size of packet
    mov %sp,%bp           # Packet pointer
    callw read            # Read from disk
    lea 0x10(%bp),%sp     # Clear stack
    lret                  # To far caller

```

stand/i386/boot2/boot1.S

Note the long return instruction at the end of this block. This instruction pops out the `%cs` register pushed by `nread`, and returns. Finally, `nread` also returns.

With the MBR loaded to memory, the actual loop for searching the FreeBSD slice begins:

```

        mov $0x1,%cx          # Two passes
main.1:
        mov $MEM_BUF+PRT_OFF,%si # Partition table
        movb $0x1,%dh          # Partition
main.2:
        cmpb $PRT_BSD,0x4(%si)  # Our partition type?
        jne main.3              # No
        jcxz main.5             # If second pass
        testb $0x80,(%si)       # Active?
        jnz main.5              # Yes
main.3:
        add $0x10,%si           # Next entry
        incb %dh                # Partition
        cmpb $0x1+PRT_NUM,%dh   # In table?
        jb main.2               # Yes
        dec %cx                 # Do two
        jcxz main.1             # passes

```

stand/i386/boot2/boot1.S

If a FreeBSD slice is identified, execution continues at `main.5`. Note that when a FreeBSD slice is found `%si` points to the appropriate entry in the partition table, and `%dh` holds the partition number. We assume that a FreeBSD slice is found, so we continue execution at `main.5`:

```

main.5:
        mov %dx,MEM_ARG          # Save args
        movb $NSECT,%dh          # Sector count
        callw nread              # Read disk
        mov $MEM_BT_X,%bx        # BTX
        mov 0xa(%bx),%si         # Get BTX length and set
        add %bx,%si              # %si to start of boot2.bin
        mov $MEM_USR+SIZ_PAG*2,%di # Client page 2
        mov $MEM_BT_X+(NSECT-1)*SIZ_SEC,%cx # Byte
        sub %si,%cx              # count
        rep                      # Relocate
        movsb                    # client

```

stand/i386/boot2/boot1.S

Recall that at this point, register `%si` points to the FreeBSD slice entry in the MBR partition table, so a call to `nread` will effectively read sectors at the beginning of this partition. The argument passed on register `%dh` tells `nread` to read 16 disk sectors. Recall that the first 512 bytes, or the first sector of the FreeBSD slice, coincides with the `boot1` program. Also recall that the file written to the beginning of the FreeBSD slice is not

/boot/boot1, but /boot/boot. Let us look at the size of these files in the filesystem:

```
-r--r--r-- 1 root wheel 512B Jan 8 00:15 /boot/boot0
-r--r--r-- 1 root wheel 512B Jan 8 00:15 /boot/boot1
-r--r--r-- 1 root wheel 7.5K Jan 8 00:15 /boot/boot2
-r--r--r-- 1 root wheel 8.0K Jan 8 00:15 /boot/boot
```

Both boot0 and boot1 are 512 bytes each, so they fit *exactly* in one disk sector. boot2 is much bigger, holding both the BTX server and the boot2 client. Finally, a file called simply boot is 512 bytes larger than boot2. This file is a concatenation of boot1 and boot2. As already noted, boot0 is the file written to the absolute first disk sector (the MBR), and boot is the file written to the first sector of the FreeBSD slice; boot1 and boot2 are *not* written to disk. The command used to concatenate boot1 and boot2 into a single boot is merely `cat boot1 boot2 > boot`.

So boot1 occupies exactly the first 512 bytes of boot and, because boot is written to the first sector of the FreeBSD slice, boot1 fits exactly in this first sector. When `nread` reads the first 16 sectors of the FreeBSD slice, it effectively reads the entire boot file ^[6]. We will see more details about how boot is formed from boot1 and boot2 in the next section.

Recall that `nread` uses memory address `0x8c00` as the transfer buffer to hold the sectors read. This address is conveniently chosen. Indeed, because boot1 belongs to the first 512 bytes, it ends up in the address range `0x8c00-0x8dff`. The 512 bytes that follows (range `0x8e00-0x8fff`) is used to store the *bsdlabel* ^[7].

Starting at address `0x9000` is the beginning of the BTX server, and immediately following is the boot2 client. The BTX server acts as a kernel, and executes in protected mode in the most privileged level. In contrast, the BTX clients (boot2, for example), execute in user mode. We will see how this is accomplished in the next section. The code after the call to `nread` locates the beginning of boot2 in the memory buffer, and copies it to memory address `0xc000`. This is because the BTX server arranges boot2 to execute in a segment starting at `0xa000`. We explore this in detail in the following section.

The last code block of boot1 enables access to memory above 1MB ^[8] and concludes with a jump to the starting point of the BTX server:

```
seta20:
    cli          # Disable interrupts
seta20.1:
    dec %cx      # Timeout?
```

```

    jz seta20.3      # Yes

    inb $0x64,%al    # Get status
    testb $0x2,%al   # Busy?
    jnz seta20.1      # Yes
    movb $0xd1,%al    # Command: Write
    outb %al,$0x64    # output port
seta20.2:
    inb $0x64,%al    # Get status
    testb $0x2,%al   # Busy?
    jnz seta20.2      # Yes
    movb $0xdf,%al    # Enable
    outb %al,$0x60    # A20
seta20.3:
    sti              # Enable interrupts
    jmp 0x9010        # Start BTX

```

stand/i386/boot2/boot1.S

Note that right before the jump, interrupts are enabled.

1.6. The BTX Server

Next in our boot sequence is the BTX Server. Let us quickly remember how we got here:

- The BIOS loads the absolute sector one (the MBR, or boot0), to address **0x7c00** and jumps there.
- boot0 relocates itself to **0x600**, the address it was linked to execute, and jumps over there. It then reads the first sector of the FreeBSD slice (which consists of boot1) into address **0x7c00** and jumps over there.
- boot1 loads the first 16 sectors of the FreeBSD slice into address **0x8c00**. This 16 sectors, or 8192 bytes, is the whole file boot. The file is a concatenation of boot1 and boot2. boot2, in turn, contains the BTX server and the boot2 client. Finally, a jump is made to address **0x9010**, the entry point of the BTX server.

Before studying the BTX Server in detail, let us further review how the single, all-in-one boot file is created. The way boot is built is defined in its Makefile (*stand/i386/boot2/Makefile*). Let us look at the rule that creates the boot file:

```

boot: boot1 boot2
cat boot1 boot2 > boot

```

stand/i386/boot2/Makefile

This tells us that boot1 and boot2 are needed, and the rule simply concatenates them to produce a single file called boot. The rules for creating boot1 are also quite simple:

```
boot1: boot1.out
${OBJCOPY} -S -O binary boot1.out ${.TARGET}

boot1.out: boot1.o
${LD} ${LD_FLAGS} -e start --defsym ORG=${ORG1} -T ${LDSCRIPT}
-o ${.TARGET} boot1.o
```

stand/i386/boot2/Makefile

To apply the rule for creating boot1, boot1.out must be resolved. This, in turn, depends on the existence of boot1.o. This last file is simply the result of assembling our familiar boot1.S, without linking. Now, the rule for creating boot1.out is applied. This tells us that boot1.o should be linked with **start** as its entry point, and starting at address **0x7c00**. Finally, boot1 is created from boot1.out applying the appropriate rule. This rule is the objcopy command applied to boot1.out. Note the flags passed to objcopy: **-S** tells it to strip all relocation and symbolic information; **-O binary** indicates the output format, that is, a simple, unformatted binary file.

Having boot1, let us take a look at how boot2 is constructed:

```
boot2: boot2.ldr
@set -- `ls -l ${.ALLSRC}`; x=$(( ${BOOT2SIZE} - $5 )); \
    echo "$$x bytes available"; test $$x -ge 0
${DD} if=${.ALLSRC} of=${.TARGET} bs=${BOOT2SIZE} conv=sync

boot2.ldr: boot2.ldr boot2.bin ${BTXKERN}
btxld -v -E ${ORG2} -f bin -b ${BTXKERN} -l boot2.ldr \
    -o ${.TARGET} -P 1 boot2.bin

boot2.ldr:
${DD} if=/dev/zero of=${.TARGET} bs=512 count=1

boot2.bin: boot2.out
${OBJCOPY} -S -O binary boot2.out ${.TARGET}

boot2.out: ${BTXCRT} boot2.o sio.o ashldi3.o
${LD} ${LD_FLAGS} --defsym ORG=${ORG2} -T ${LDSCRIPT} -o
${.TARGET} ${.ALLSRC}

boot2.h: boot1.out
```

```

${NM} -t d ${.ALLSRC} | awk '/([0-9])+ T xread/ \
{ x = $$1 - ORG1; \
printf("#define XREADORG %#x\n", REL1 + x) }' \
ORG1=`printf "%d" ${ORG1}` \
REL1=`printf "%d" ${REL1}` > ${.TARGET}

```

stand/i386/boot2/Makefile

The mechanism for building boot2 is far more elaborate. Let us point out the most relevant facts. The dependency list is as follows:

```

boot2: boot2.ld
boot2.ld: boot2.ldr boot2.bin ${BTXDIR}
boot2.bin: boot2.out
boot2.out: ${BTXDIR} boot2.o sio.o ashldi3.o
boot2.h: boot1.out

```

stand/i386/boot2/Makefile

Note that initially there is no header file boot2.h, but its creation depends on boot1.out, which we already have. The rule for its creation is a bit terse, but the important thing is that the output, boot2.h, is something like this:

```
#define XREADORG 0x725
```

stand/i386/boot2/boot2.h

Recall that boot1 was relocated (i.e., copied from **0x7c00** to **0x700**). This relocation will now make sense, because as we will see, the BTX server reclaims some memory, including the space where boot1 was originally loaded. However, the BTX server needs access to boot1's **xread** function; this function, according to the output of boot2.h, is at location **0x725**. Indeed, the BTX server uses the **xread** function from boot1's relocated code. This function is now accessible from within the boot2 client.

The next rule directs the linker to link various files (ashldi3.o, boot2.o and sio.o). Note that the output file, boot2.out, is linked to execute at address **0x2000** (**\${ORG2}**). Recall that boot2 will be executed in user mode, within a special user segment set up by the BTX server. This segment starts at **0xa000**. Also, remember that the boot2 portion of boot was copied to address **0xc000**, that is, offset **0x2000** from the start of the user segment, so boot2 will work properly when we transfer control to it. Next, boot2.bin is created from boot2.out by stripping its symbols and format information; boot2.bin is a *raw* binary. Now, note that a file boot2.ldr is created as a 512-byte file full of zeros. This space is

reserved for the `bsdlabel`.

Now that we have files `boot1`, `boot2.bin` and `boot2.ldr`, only the BTX server is missing before creating the all-in-one boot file. The BTX server is located in `stand/i386/btx/btx`; it has its own Makefile with its own set of rules for building. The important thing to notice is that it is also compiled as a *raw* binary, and that it is linked to execute at address `0x9000`. The details can be found in `stand/i386/btx/btx/Makefile`.

Having the files that comprise the boot program, the final step is to *merge* them. This is done by a special program called `btxld` (source located in `/usr/src/usr.sbin/btxld`). Some arguments to this program include the name of the output file (`boot`), its entry point (`0x2000`) and its file format (raw binary). The various files are finally merged by this utility into the file `boot`, which consists of `boot1`, `boot2`, the `bsdlabel` and the BTX server. This file, which takes exactly 16 sectors, or 8192 bytes, is what is actually written to the beginning of the FreeBSD slice during installation. Let us now proceed to study the BTX server program.

The BTX server prepares a simple environment and switches from 16-bit real mode to 32-bit protected mode, right before passing control to the client. This includes initializing and updating the following data structures:

- Modifies the **Interrupt Vector Table (IVT)**. The IVT provides exception and interrupt handlers for Real-Mode code.
- The **Interrupt Descriptor Table (IDT)** is created. Entries are provided for processor exceptions, hardware interrupts, two system calls and V86 interface. The IDT provides exception and interrupt handlers for Protected-Mode code.
- A **Task-State Segment (TSS)** is created. This is necessary because the processor works in the *least* privileged level when executing the client (`boot2`), but in the *most* privileged level when executing the BTX server.
- The GDT (Global Descriptor Table) is set up. Entries (descriptors) are provided for supervisor code and data, user code and data, and real-mode code and data. ^[9]

Let us now start studying the actual implementation. Recall that `boot1` made a jump to address `0x9010`, the BTX server's entry point. Before studying program execution there, note that the BTX server has a special header at address range `0x9000-0x900f`, right before its entry point. This header is defined as follows:

```
start:                                # Start of code
```

```

/*
 * BTX header.
 */
btx_hdr:    .byte 0xeb          # Machine ID
            .byte 0xe          # Header size
            .ascii "BTX"        # Magic
            .byte 0x1          # Major version
            .byte 0x2          # Minor version
            .byte BTX_FLAGS     # Flags
            .word PAG_CNT-MEM_ORG>>0xc    # Paging control
            .word break-start   # Text size
            .long 0x0           # Entry address

```

stand/i386/btx/btx/btx.S

Note the first two bytes are **0xeb** and **0xe**. In the IA-32 architecture, these two bytes are interpreted as a relative jump past the header into the entry point, so in theory, boot1 could jump here (address **0x9000**) instead of address **0x9010**. Note that the last field in the BTX header is a pointer to the client's (boot2) entry pointb2. This field is patched at link time.

Immediately following the header is the BTX server's entry point:

```

/*
 * Initialization routine.
 */
init:      cli                # Disable interrupts
            xor %ax,%ax        # Zero/segment
            mov %ax,%ss        # Set up
            mov $MEM_ESP0,%sp   # stack
            mov %ax,%es        # Address
            mov %ax,%ds        # data
            pushl $0x2          # Clear
            popfl              # flags

```

stand/i386/btx/btx/btx.S

This code disables interrupts, sets up a working stack (starting at address **0x1800**) and clears the flags in the EFLAGS register. Note that the **popfl** instruction pops out a doubleword (4 bytes) from the stack and places it in the EFLAGS register. As the value actually popped is **2**, the EFLAGS register is effectively cleared (IA-32 requires that bit 2 of the EFLAGS register always be 1).

Our next code block clears (sets to **0**) the memory range **0x5e00-0x8fff**. This range is where the various data structures will be created:

```

/*
 * Initialize memory.
 */
    mov $MEM_IDT,%di      # Memory to initialize
    mov $(MEM_ORG-MEM_IDT)/2,%cx  # Words to zero
    rep                   # Zero-fill
    stosw                  # memory

```

stand/i386/btx/btx/btx.S

Recall that `boot1` was originally loaded to address `0x7c00`, so, with this memory initialization, that copy effectively disappeared. However, also recall that `boot1` was relocated to `0x700`, so *that* copy is still in memory, and the BTX server will make use of it.

Next, the real-mode IVT (Interrupt Vector Table) is updated. The IVT is an array of segment/offset pairs for exception and interrupt handlers. The BIOS normally maps hardware interrupts to interrupt vectors `0x8` to `0xf` and `0x70` to `0x77` but, as will be seen, the 8259A Programmable Interrupt Controller, the chip controlling the actual mapping of hardware interrupts to interrupt vectors, is programmed to remap these interrupt vectors from `0x8-0xf` to `0x20-0x27` and from `0x70-0x77` to `0x28-0x2f`. Thus, interrupt handlers are provided for interrupt vectors `0x20-0x2f`. The reason the BIOS-provided handlers are not used directly is because they work in 16-bit real mode, but not 32-bit protected mode. Processor mode will be switched to 32-bit protected mode shortly. However, the BTX server sets up a mechanism to effectively use the handlers provided by the BIOS:

```

/*
 * Update real mode IDT for reflecting hardware interrupts.
 */
    mov $intr20,%bx      # Address first handler
    mov $0x10,%cx        # Number of handlers
    mov $0x20*4,%di      # First real mode IDT entry
init.0:    mov %bx, (%di)  # Store IP
    inc %di              # Address next
    inc %di              # entry
    stosw                # Store CS
    add $4,%bx           # Next handler
    loop init.0          # Next IRQ

```

stand/i386/btx/btx/btx.S

The next block creates the IDT (Interrupt Descriptor Table). The IDT is analogous, in

protected mode, to the IVT in real mode. That is, the IDT describes the various exception and interrupt handlers used when the processor is executing in protected mode. In essence, it also consists of an array of segment/offset pairs, although the structure is somewhat more complex, because segments in protected mode are different than in real mode, and various protection mechanisms apply:

```

/*
 * Create IDT.
 */
    mov $MEM_IDT,%di      # IDT's address
    mov $idtctl,%si       # Control string
init.1:    lodsb           # Get entry
           cbw            # count
           xchg %ax,%cx    # as word
           jcxz init.4     # If done
           lodsb          # Get segment
           xchg %ax,%dx    # P:DPL:type
           lodsw          # Get control
           xchg %ax,%bx    # set
           lodsw          # Get handler offset
           mov $SEL_SCOPE,%dh # Segment selector
init.2:    shr %bx         # Handle this int?
           jnc init.3      # No
           mov %ax, (%di)  # Set handler offset
           mov %dh, 0x2(%di) # and selector
           mov %dl, 0x5(%di) # Set P:DPL:type
           add $0x4,%ax    # Next handler
init.3:    lea 0x8(%di),%di # Next entry
           loop init.2     # Till set done
           jmp init.1      # Continue

```

stand/i386/btx/btx.S

Each entry in the **IDT** is 8 bytes long. Besides the segment/offset information, they also describe the segment type, privilege level, and whether the segment is present in memory or not. The construction is such that interrupt vectors from **0** to **0xf** (exceptions) are handled by function **intx00**; vector **0x10** (also an exception) is handled by **intx10**; hardware interrupts, which are later configured to start at interrupt vector **0x20** all the way to interrupt vector **0x2f**, are handled by function **intx20**. Lastly, interrupt vector **0x30**, which is used for system calls, is handled by **intx30**, and vectors **0x31** and **0x32** are handled by **intx31**. It must be noted that only descriptors for interrupt vectors **0x30**, **0x31** and **0x32** are given privilege level 3, the same privilege level as the boot2 client, which means the client can execute a software-generated interrupt to this vectors through the **int** instruction without failing (this is the way boot2 use the services provided by the

BTX server). Also, note that *only* software-generated interrupts are protected from code executing in lesser privilege levels. Hardware-generated interrupts and processor-generated exceptions are *always* handled adequately, regardless of the actual privileges involved.

The next step is to initialize the TSS (Task-State Segment). The TSS is a hardware feature that helps the operating system or executive software implement multitasking functionality through process abstraction. The IA-32 architecture demands the creation and use of *at least* one TSS if multitasking facilities are used or different privilege levels are defined. Since the boot2 client is executed in privilege level 3, but the BTX server runs in privilege level 0, a TSS must be defined:

```
/*
 * Initialize TSS.
 */
init.4:      movb $_ESP0H, TSS_ESP0+1(%di)    # Set ESP0
             movb $SEL_SDATA, TSS_SS0(%di)    # Set SS0
             movb $_TSSIO, TSS_MAP(%di)      # Set I/O bit map base
```

stand/i386/btx/btx.S

Note that a value is given for the Privilege Level 0 stack pointer and stack segment in the TSS. This is needed because, if an interrupt or exception is received while executing boot2 in Privilege Level 3, a change to Privilege Level 0 is automatically performed by the processor, so a new working stack is needed. Finally, the I/O Map Base Address field of the TSS is given a value, which is a 16-bit offset from the beginning of the TSS to the I/O Permission Bitmap and the Interrupt Redirection Bitmap.

After the IDT and TSS are created, the processor is ready to switch to protected mode. This is done in the next block:

```
/*
 * Bring up the system.
 */
             mov $0x2820, %bx                # Set protected mode
             callw setpic                     # IRQ offsets
             lidt idtdesc                     # Set IDT
             lgdt gtdesc                      # Set GDT
             mov %cr0, %eax                   # Switch to protected
             inc %ax                          # mode
             mov %eax, %cr0                   #
             ljmp $SEL_SCODE, $init.8        # To 32-bit code
```

```

        .code32
init.8:  xorl %ecx,%ecx          # Zero
        movb $SEL_SDATA,%cl    # To 32-bit
        movw %cx,%ss           # stack

```

stand/i386/btx/btx/btx.S

First, a call is made to `setpic` to program the 8259A PIC (Programmable Interrupt Controller). This chip is connected to multiple hardware interrupt sources. Upon receiving an interrupt from a device, it signals the processor with the appropriate interrupt vector. This can be customized so that specific interrupts are associated with specific interrupt vectors, as explained before. Next, the IDTR (Interrupt Descriptor Table Register) and GDTR (Global Descriptor Table Register) are loaded with the instructions `lidt` and `lgdt`, respectively. These registers are loaded with the base address and limit address for the IDT and GDT. The following three instructions set the Protection Enable (PE) bit of the `%cr0` register. This effectively switches the processor to 32-bit protected mode. Next, a long jump is made to `init.8` using segment selector `SEL_SCODE`, which selects the Supervisor Code Segment. The processor is effectively executing in CPL 0, the most privileged level, after this jump. Finally, the Supervisor Data Segment is selected for the stack by assigning the segment selector `SEL_SDATA` to the `%ss` register. This data segment also has a privilege level of 0.

Our last code block is responsible for loading the TR (Task Register) with the segment selector for the TSS we created earlier, and setting the User Mode environment before passing execution control to the boot2 client.

```

/*
 * Launch user task.
 */
        movb $SEL_TSS,%cl      # Set task
        ltr %cx                # register
        movl $MEM_USR,%edx     # User base address
        movzwl %ss:BDA_MEM,%eax # Get free memory
        shll $0xa,%eax         # To bytes
        subl $ARGSPACE,%eax    # Less arg space
        subl %edx,%eax         # Less base
        movb $SEL_UDATA,%cl    # User data selector
        pushl %ecx              # Set SS
        pushl %eax              # Set ESP
        push $0x202            # Set flags (IF set)
        push $SEL_UCODE        # Set CS
        pushl btx_hdr+0xc      # Set EIP
        pushl %ecx              # Set GS
        pushl %ecx              # Set FS

```

```

        pushl %ecx          # Set DS
        pushl %ecx          # Set ES
        pushl %edx          # Set EAX
        movb $0x7,%cl      # Set remaining
init.9:   push $0x0         # general
        loop init.9        # registers
#ifdef BTX_SERIAL
        call sio_init      # setup the serial console
#endif
        popa               # and initialize
        popl %es           # Initialize
        popl %ds           # user
        popl %fs           # segment
        popl %gs           # registers
        iret               # To user mode

```

stand/i386/btx/btx/btx.S

Note that the client's environment include a stack segment selector and stack pointer (registers `%ss` and `%esp`). Indeed, once the TR is loaded with the appropriate stack segment selector (instruction `ltr`), the stack pointer is calculated and pushed onto the stack along with the stack's segment selector. Next, the value `0x202` is pushed onto the stack; it is the value that the EFLAGS will get when control is passed to the client. Also, the User Mode code segment selector and the client's entry point are pushed. Recall that this entry point is patched in the BTX header at link time. Finally, segment selectors (stored in register `%ecx`) for the segment registers `%gs`, `%fs`, `%ds` and `%es` are pushed onto the stack, along with the value at `%edx` (`0xa000`). Keep in mind the various values that have been pushed onto the stack (they will be popped out shortly). Next, values for the remaining general purpose registers are also pushed onto the stack (note the `loop` that pushes the value `0` seven times). Now, values will be started to be popped out of the stack. First, the `popa` instruction pops out of the stack the latest seven values pushed. They are stored in the general purpose registers in order `%edi`, `%esi`, `%ebp`, `%ebx`, `%edx`, `%ecx`, `%eax`. Then, the various segment selectors pushed are popped into the various segment registers. Five values still remain on the stack. They are popped when the `iret` instruction is executed. This instruction first pops the value that was pushed from the BTX header. This value is a pointer to boot2's entry point. It is placed in the register `%eip`, the instruction pointer register. Next, the segment selector for the User Code Segment is popped and copied to register `%cs`. Remember that this segment's privilege level is 3, the least privileged level. This means that we must provide values for the stack of this privilege level. This is why the processor, besides further popping the value for the EFLAGS register, does two more pops out of the stack. These values go to the stack pointer (`%esp`) and the stack segment (`%ss`). Now, execution continues at `boot0`'s entry point.

It is important to note how the User Code Segment is defined. This segment's *base address* is set to `0xa000`. This means that code memory addresses are *relative* to address `0xa000`; if code being executed is fetched from address `0x2000`, the *actual* memory addressed is `0xa000+0x2000=0xc000`.

1.7. boot2 Stage

`boot2` defines an important structure, `struct bootinfo`. This structure is initialized by `boot2` and passed to the loader, and then further to the kernel. Some nodes of this structures are set by `boot2`, the rest by the loader. This structure, among other information, contains the kernel filename, BIOS harddisk geometry, BIOS drive number for boot device, physical memory available, `envp` pointer etc. The definition for it is:

```
/usr/include/machine/bootinfo.h:
struct bootinfo {
    u_int32_t    bi_version;
    u_int32_t    bi_kernelname;      /* represents a char * */
    u_int32_t    bi_nfs_diskless;    /* struct nfs_diskless * */
    /* End of fields that are always present. */
#define bi_endcommon    bi_n_bios_used
    u_int32_t    bi_n_bios_used;
    u_int32_t    bi_bios_geom[N_BIOS_GEOM];
    u_int32_t    bi_size;
    u_int8_t     bi_memsizes_valid;
    u_int8_t     bi_bios_dev;        /* bootdev BIOS unit number */
    u_int8_t     bi_pad[2];
    u_int32_t    bi_basemem;
    u_int32_t    bi_extmem;
    u_int32_t    bi_syntab;          /* struct syntab * */
    u_int32_t    bi_esyntab;         /* struct syntab * */
    /* Items below only from advanced bootloader */
    u_int32_t    bi_kernend;         /* end of kernel space */
    u_int32_t    bi_envp;            /* environment */
    u_int32_t    bi_modulep;        /* preloaded modules */
};
```

`boot2` enters into an infinite loop waiting for user input, then calls `load()`. If the user does not press anything, the loop breaks by a timeout, so `load()` will load the default file (`/boot/loader`). Functions `ino_t lookup(char *filename)` and `int xfsread(ino_t inode, void *buf, size_t nbyte)` are used to read the content of a file into memory. `/boot/loader` is an ELF binary, but where the ELF header is prepended with `a.out`'s `struct exec` structure. `load()` scans the loader's ELF header, loading the content of `/boot/loader` into memory, and passing the execution to the loader's entry:

```
stand/i386/boot2/boot2.c:
    __exec((caddr_t)addr, RB_BOOTINFO | (opts & RBX_MASK),
          MAKEBOOTDEV(dev_maj[dsk.type], dsk.slice, dsk.unit,
dsk.part),
          0, 0, 0, VTOP(&bootinfo));
```

1.8. loader Stage

loader is a BTX client as well. I will not describe it here in detail, there is a comprehensive man page written by Mike Smith, `man:loader[8]`. The underlying mechanisms and BTX were discussed above.

The main task for the loader is to boot the kernel. When the kernel is loaded into memory, it is being called by the loader:

```
stand/common/boot.c:
    /* Call the exec handler from the loader matching the kernel
    */
    file_formats[fp->f_loader]->l_exec(fp);
```

1.9. Kernel Initialization

Let us take a look at the command that links the kernel. This will help identify the exact location where the loader passes execution to the kernel. This location is the kernel's actual entry point. This command is now excluded from `sys/conf/Makefile.i386`. The content that interests us can be found in `/usr/obj/usr/src/i386.i386/sys/GENERIC/`.

```
/usr/obj/usr/src/i386.i386/sys/GENERIC/kernel.meta:
ld -m elf_i386_fbsd -Bdynamic -T /usr/src/sys/conf/ldscript.i386
--build-id=sha1 --no-warn-mismatch \
--warn-common --export-dynamic --dynamic-linker /red/herring -X
-o kernel locore.o
<lots of kernel .o files>
```

A few interesting things can be seen here. First, the kernel is an ELF dynamically linked binary, but the dynamic linker for kernel is `/red/herring`, which is definitely a bogus file. Second, taking a look at the file `sys/conf/ldscript.i386` gives an idea about what ld options are used when compiling a kernel. Reading through the first few lines, the string

```
sys/conf/ldscript.i386:
ENTRY(btext)
```

says that a kernel's entry point is the symbol `btext`. This symbol is defined in `locore.s`:

```
sys/i386/i386/locore.s:
    .text
    /*****
    ****
    *
    * This is where the bootblocks start us, set the ball rolling...
    *
    */
    NON_GPROF_ENTRY(btext)
```

First, the register `EFLAGS` is set to a predefined value of `0x00000002`. Then all the segment registers are initialized:

```
sys/i386/i386/locore.s:
/* Don't trust what the BIOS gives for eflags. */
    pushl    $PSL_KERNEL
    popfl

/*
 * Don't trust what the BIOS gives for %fs and %gs. Trust the
 * bootstrap
 * to set %cs, %ds, %es and %ss.
 */
    mov %ds, %ax
    mov %ax, %fs
    mov %ax, %gs
```

`btext` calls the routines `recover_bootinfo()`, `identify_cpu()`, which are also defined in `locore.s`. Here is a description of what they do: